

src 代码逐个讲解 - 零基础阅读版

这份 PDF 包含 src 文件夹中的全部 Python 代码。每个文件前先解释用途和阅读重点，再给出带行号的完整源码。建议按 `__init__.py` → `types.py` → `store.py` → `tools.py` → `llm.py` → `tasks.py` → `runtime.py` 的顺序阅读。

阅读方法

先不要逐字理解。每个文件先回答三个问题：它负责什么？它从谁那里拿数据？它把结果交给谁？看不懂的英文函数名可以先跳过，先盯住中文注释、类名和函数名。

核心流程

`main.py` 把用户问题交给 `runtime.py`；`runtime.py` 从 `store.py` 取记忆，交给 `llm.py` 请求 DeepSeek；如果模型选择工具，就用 `tools.py` 执行；异步工具由 `tasks.py` 在后台完成，并把结果事件写回 `store.py`。

__init__.py

它负责什么：包标记文件。它告诉 Python：src 是一个可以被导入的代码包。

建议怎么读：内容很短，只需要知道有它以后，main.py 才能写 `from src.runtime import AgentRuntime`。

本文件最重要的理解：它本身不放业务逻辑，作用是组织项目目录。

完整代码（带行号）

```
1 """Minimal Agent runtime package."""  
2
```

types.py

它负责什么：数据合同。它规定模型回复、工具调用结果、最终 Agent 结果分别长什么样。

建议怎么读：先认识三个 @dataclass：ToolCall 是一次工具请求；LLMReply 是模型的一次回复；AgentResult 是 Runtime 最终交给 main.py 的结果。

本文件最重要的理解：把它理解为三张固定格式的表格。其他文件只要遵循这些格式，就能协同工作。

完整代码（带行号）

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass, field
4 from typing import Any
5
6
7 @dataclass
8 class ToolCall:
9     id: str
10    name: str
11    arguments: dict[str, Any]
12
13
14 @dataclass
15 class LLMReply:
16     content: str | None = None
17     tool_calls: list[ToolCall] = field(default_factory=list)
18
19
20 @dataclass
21 class AgentResult:
22     session_id: str
23     answer: str
24     trace: list[dict[str, Any]]
25     stopped_by_limit: bool = False
26
```

store.py

它负责什么：本地记忆本。它把 session 短期上下文、用户长期记忆、待办和任务事件保存进 JSON 文件。

建议怎么读：重点看 session()、remember()/recall() 和 claim_session()/claim_next_input()。前者区分短期与长期记忆，后者实现 busy 队列。

本文件最重要的理解：user_id 决定谁的长期记忆；user_id + session_id 决定哪个聊天窗口的数据。

完整代码（带行号）

```
1 from __future__ import annotations
2
3 import json
4 from pathlib import Path
5 from threading import RLock
6 from typing import Any
7
8
9 class JsonStore:
10     """Inspectable local persistence. Every session key is scoped by user and session."""
11
12     def __init__(self, path: str | Path = "data/agent_state.json") -> None:
13         self.path = Path(path)
14         self.lock = RLock()
15         self.data: dict[str, Any] = {"sessions": {}, "user_memories": {}, "todos": {}}
16         if self.path.exists():
17             self.data.update(json.loads(self.path.read_text(encoding="utf-8")))
18
19     def _save(self) -> None:
20         self.path.parent.mkdir(parents=True, exist_ok=True)
21         self.path.write_text(json.dumps(self.data, ensure_ascii=False, indent=2), encoding="utf-8")
22
23     def session(self, user_id: str, session_id: str) -> dict[str, Any]:
24         # session 短期上下文的隔离边界：同一 user 的不同窗口也不会串数据。
25         key = f"{user_id}:{session_id}"
26         with self.lock:
27             session = self.data["sessions"].setdefault(key, {"user_id": user_id, "session_id": session_id, "summary": "",
28 > "messages": []})
29             # Migration defaults for data written by v1.
30             session.setdefault("status", "idle")
31             session.setdefault("pending_inputs", [])
32             session.setdefault("events", [])
33         return session
34
35     def save_session(self, session: dict[str, Any]) -> None:
36         with self.lock:
```

```

36     self.data["sessions"][f"{session['user_id']}:{session['session_id']}"] = session
37     self._save()
38
39 def claim_session(self, user_id: str, session_id: str, user_input: str) -> bool:
40     """Claim an idle session; otherwise persist the new message in its FIFO queue."""
41     with self.lock:
42         session = self.session(user_id, session_id)
43         if session["status"] == "busy":
44             # FIFO 队列仅保存用户输入；执行时再按当时最新 Context 调用模型。
45             session["pending_inputs"].append(user_input)
46             self._save()
47             return False
48         session["status"] = "busy"
49         self._save()
50         return True
51
52 def finish_session(self, user_id: str, session_id: str) -> None:
53     with self.lock:
54         session = self.session(user_id, session_id)
55         session["status"] = "idle"
56         self._save()
57
58 def claim_next_input(self, user_id: str, session_id: str) -> str | None:
59     with self.lock:
60         session = self.session(user_id, session_id)
61         if session["status"] != "idle" or not session["pending_inputs"]:
62             return None
63         session["status"] = "busy"
64         value = session["pending_inputs"].pop(0)
65         self._save()
66         return value
67
68 def add_event(self, user_id: str, session_id: str, event: dict[str, Any]) -> None:
69     with self.lock:
70         self.session(user_id, session_id)["events"].append(event)
71         self._save()
72
73 def list_events(self, user_id: str, session_id: str, clear: bool = False) -> list[dict[str, Any]]:
74     with self.lock:
75         events = list(self.session(user_id, session_id)["events"])
76         if clear:
77             self.session(user_id, session_id)["events"] = []
78             self._save()
79         return events
80
81 def remember(self, user_id: str, text: str) -> None:
82     # 长期记忆只按 user_id 索引，因此可以跨 session 共享。
83     with self.lock:
84         memories = self.data["user_memories"].setdefault(user_id, [])

```

```
85         if text not in memories:
86             memories.append(text)
87             self._save()
88
89     def recall(self, user_id: str, query: str, top_k: int = 3) -> list[str]:
90         # 第一/二版采用可解释的字符匹配; 生产版可替换为 embedding 语义检索。
91         characters = {ch.lower() for ch in query if not ch.isspace()}
92         memories = self.data["user_memories"].get(user_id, [])
93         ranked = sorted(memories, key=lambda item: sum(ch.lower() in characters for ch in item), reverse=True)
94         return ranked[:top_k]
95
96     def add_todo(self, user_id: str, session_id: str, task: str) -> dict[str, Any]:
97         with self.lock:
98             key = f"{user_id}:{session_id}"
99             todos = self.data["todos"].setdefault(key, [])
100             item = {"id": len(todos) + 1, "task": task, "done": False}
101             todos.append(item)
102             self._save()
103             return item
104
105     def list_todos(self, user_id: str, session_id: str) -> list[dict[str, Any]]:
106         return self.data["todos"].get(f"{user_id}:{session_id}", [])
```

tools.py

它负责什么：工具箱和工具管理员。模型只能从这里注册过的工具中选择。

建议怎么读：先看 Tool 的四项信息，再看 ToolRegistry 的 register、schemas、execute；最后看 calculator、todo 和异步任务工具如何被注册。

本文件最重要的理解：工具 Schema 是给模型看的说明书；validate 是 Runtime 执行前的安全检查。

完整代码（带行号）

```
1 from __future__ import annotations
2
3 import ast
4 import operator
5 from dataclasses import dataclass
6 from typing import Any, Callable
7
8 from .store import JsonStore
9 from .tasks import AsyncTaskManager, delayed_search
10
11
12 @dataclass
13 class Tool:
14     name: str
15     description: str
16     parameters: dict[str, Any]
17     handler: Callable[[dict[str, Any], dict[str, str]], Any]
18
19     def schema(self) -> dict[str, Any]:
20         return {"type": "function", "function": {"name": self.name, "description": self.description, "parameters":
21             > self.parameters}}
22
23     def validate(self, args: dict[str, Any]) -> None:
24         # 在执行函数前拦截缺少必填项和 enum 非法值，错误可回传给模型修正。
25         if not isinstance(args, dict):
26             raise ValueError("tool arguments must be an object")
27         for field in self.parameters.get("required", []):
28             if field not in args or args[field] in (None, ""):
29                 raise ValueError(f"missing required argument: {field}")
30         for field, spec in self.parameters.get("properties", {}).items():
31             if field in args and "enum" in spec and args[field] not in spec["enum"]:
32                 raise ValueError(f"invalid {field}: {args[field]}")
33
34 class ToolRegistry:
35     def __init__(self) -> None:
```

```

36     self._tools: dict[str, Tool] = {}
37
38     def register(self, tool: Tool) -> None:
39         if tool.name in self._tools:
40             raise ValueError(f"tool already registered: {tool.name}")
41         self._tools[tool.name] = tool
42
43     def schemas(self) -> list[dict[str, Any]]:
44         return [tool.schema() for tool in self._tools.values()]
45
46     def execute(self, name: str, args: dict[str, Any], context: dict[str, str]) -> Any:
47         if name not in self._tools:
48             raise ValueError(f"unknown tool: {name}")
49         tool = self._tools[name]
50         tool.validate(args)
51         return tool.handler(args, context)
52
53
54 _OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul, ast.Div: operator.truediv, ast.USub:
> operator.neg}
55
56
57 def _calculate(expression: str) -> float:
58     # AST 白名单代替 eval, 禁止导入模块、属性访问和任意代码执行。
59     def visit(node: ast.AST) -> float:
60         if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
61             return node.value
62         if isinstance(node, ast.BinOp) and type(node.op) in _OPS:
63             return _OPS[type(node.op)](visit(node.left), visit(node.right))
64         if isinstance(node, ast.UnaryOp) and type(node.op) in _OPS:
65             return _OPS[type(node.op)](visit(node.operand))
66         raise ValueError("only +, -, *, / and parentheses are allowed")
67     return visit(ast.parse(expression, mode="eval").body)
68
69
70 def build_default_registry(store: JsonStore, task_manager: AsyncTaskManager | None = None) -> ToolRegistry:
71     registry = ToolRegistry()
72     registry.register(Tool("calculator", "Evaluate a basic arithmetic expression.", {"type": "object", "properties":
> {"expression": {"type": "string"}}, "required": ["expression"]}, lambda a, _: {"result": _calculate(a["expression"])}))
73     registry.register(Tool("search", "Search a mock knowledge base.", {"type": "object", "properties": {"query": {"type":
> "string"}}, "required": ["query"]}, lambda a, _: {"query": a["query"], "results": [f"Mock result for: {a['query']}"]}))
74
75     def todo(args: dict[str, Any], ctx: dict[str, str]) -> Any:
76         if args["action"] == "add":
77             if not args.get("task"):
78                 raise ValueError("task is required when action is add")
79             return store.add_todo(ctx["user_id"], ctx["session_id"], args["task"])
80         return store.list_todos(ctx["user_id"], ctx["session_id"])
81     registry.register(Tool("todo", "Add or list todos belonging to the current session.", {"type": "object", "properties":

```



```

> {"action": {"type": "string", "enum": ["add", "list"]}, "task": {"type": "string"}}, "required": ["action"]}, todo))
82
83     if task_manager:
84         # 只有传入任务管理器时才暴露异步工具，保持第一版核心的最小性。
85         def background_search(args: dict[str, Any], ctx: dict[str, str]) -> Any:
86             delay = float(args.get("delay_seconds", 0.2))
87             return task_manager.submit(ctx["user_id"], ctx["session_id"], "background_search", lambda cancel:
88 > delayed_search(args["query"], delay, cancel))
89         registry.register(Tool("background_search", "Start an asynchronous mock search; it returns a task id immediately.",
90 > {"type": "object", "properties": {"query": {"type": "string"}, "delay_seconds": {"type": "number"}}, "required": ["query"]},
91 > background_search))
92
93     def task_control(args: dict[str, Any], ctx: dict[str, str]) -> Any:
94         if args["action"] == "cancel":
95             return task_manager.cancel(args["task_id"], ctx["user_id"], ctx["session_id"])
96             return task_manager.status(args["task_id"], ctx["user_id"], ctx["session_id"])
97         registry.register(Tool("task_control", "Check or cancel an asynchronous task in the current session.", {"type":
98 > "object", "properties": {"action": {"type": "string", "enum": ["status", "cancel"]}, "task_id": {"type": "string"}},
99 > "required": ["action", "task_id"]}, task_control))
100     return registry

```

llm.py

它负责什么：DeepSeek 通信层。它只负责把消息发给 API、拿回回复，并解析出工具调用。

建议怎么读：_request 是最底层 HTTP 请求；chat 用于正常对话；summarize 用于生成结构化摘要；ToolCallParser 解析模型输出。

本文件最重要的理解：这层没有 Agent Loop。真正决定“继续调用工具还是结束”的逻辑在 runtime.py。

完整代码（带行号）

```
1 from __future__ import annotations
2
3 import json
4 import os
5 import re
6 import urllib.error
7 import urllib.request
8 from typing import Any
9
10 from .types import LLMReply, ToolCall
11
12
13 class DeepSeekClient:
14     """Small OpenAI-compatible client; the Agent loop remains implemented locally."""
15
16     def __init__(self, api_key: str | None = None, base_url: str | None = None, model: str | None = None) -> None:
17         self.api_key = api_key or os.getenv("DEEPSEEK_API_KEY")
18         self.base_url = (base_url or os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com")).rstrip("/")
19         self.model = model or os.getenv("DEEPSEEK_MODEL", "deepseek-chat")
20
21     def _request(self, messages: list[dict[str, Any]], tools: list[dict[str, Any]] | None = None) -> dict[str, Any]:
22         if not self.api_key:
23             raise RuntimeError("Missing DEEPSEEK_API_KEY. Copy .env.example values into your environment.")
24         payload: dict[str, Any] = {"model": self.model, "messages": messages, "temperature": 0.2}
25         if tools is not None:
26             # 传入工具 Schema 后，DeepSeek 才能返回 OpenAI-compatible tool_calls。
27             payload.update({"tools": tools, "tool_choice": "auto"})
28         request = urllib.request.Request(f"{self.base_url}/chat/completions", data=json.dumps(payload).encode(),
29 > headers={"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"})
30         try:
31             with urllib.request.urlopen(request, timeout=60) as response:
32                 return json.loads(response.read())
33         except urllib.error.HTTPError as exc:
34             raise RuntimeError(f"DeepSeek API failed ({exc.code}): {exc.read().decode(errors='replace')}") from exc
35
36     def chat(self, messages: list[dict[str, Any]], tools: list[dict[str, Any]] -> LLMReply:
```

```

36     return ToolCallParser.parse(self._request(messages, tools)[0][0]["message"])
37
38     def summarize(self, previous_summary: str, old_messages: list[dict[str, Any]]) -> str:
39         # 摘要请求不传工具，避免模型在压缩历史时触发业务工具调用。
40         source = [{"role": item["role"], "content": item.get("content", "")[:500]} for item in old_messages if item["role"] in
41             > {"user", "assistant", "tool"}]
42         prompt = ("Compress the conversation into JSON only. Required keys: goals, conclusions, open_items, preferences. "
43             "Each value must be a JSON array of concise strings. Preserve only durable task state; do not invent
44             > facts.\n"
45             f"Previous summary: {previous_summary}\nMessages: {json.dumps(source, ensure_ascii=False)}")
46         content = self._request([{"role": "system", "content": "You produce valid JSON and nothing else."}, {"role": "user",
47             > "content": prompt}])[0][0]["message"].get("content", "{}")
48         return StructuredSummary.normalise(content)
49
50 class StructuredSummary:
51     KEYS = ("goals", "conclusions", "open_items", "preferences")
52
53     @classmethod
54     def normalise(cls, text: str) -> str:
55         # 校验并补齐 JSON 键，保证 Runtime 后续拿到稳定的摘要结构。
56         clean = re.sub(r"^(?:(?:json)?\s*\$)", "", text.strip(), flags=re.S)
57         value = json.loads(clean)
58         if not isinstance(value, dict):
59             raise ValueError("summary is not a JSON object")
60         result = {key: [str(item) for item in value.get(key, [])[:8]] for key in cls.KEYS}
61         return json.dumps(result, ensure_ascii=False)
62
63 class ToolCallParser:
64     @staticmethod
65     def parse(message: dict[str, Any]) -> LLMReply:
66         calls = []
67         for call in message.get("tool_calls") or []:
68             # 标准 function calling：解析模型给出的工具名和 JSON 参数。
69             fn = call["function"]
70             calls.append(ToolCall(call.get("id", f"call_{len(calls)}"), fn["name"], json.loads(fn.get("arguments") or "{}")))
71         content = message.get("content") or ""
72         if not calls:
73             # 兼容模型偶尔以 Markdown JSON 而非原生 tool_calls 输出的情况。
74             match = re.search(r"^(?:(?:json)?\s*\{.*?\})\s*$", content, re.S)
75             if match:
76                 try:
77                     item = json.loads(match.group(1))
78                     if "tool" in item:
79                         calls.append(ToolCall("text_call_0", item["tool"], item.get("arguments", {})))
80                 except json.JSONDecodeError:
81                     pass
82         return LLMReply(content=content, tool_calls=calls)

```

tasks.py

它负责什么：后台任务管理员。它让耗时工具先返回 task_id，在后台执行，完成后向所属 session 写入事件。

建议怎么读：submit 创建任务并放入线程池；complete 处理完成/失败/取消；cancel 发送协作式取消信号。

本文件最重要的理解：Python 不适合强杀运行中线程，所以任务自己定期检查 cancel_event 后退出。

完整代码（带行号）

```
1 from __future__ import annotations
2
3 from concurrent.futures import Future, ThreadPoolExecutor
4 from dataclasses import dataclass
5 from threading import Event, RLock
6 from time import sleep, time
7 from typing import Any, Callable
8 from uuid import uuid4
9
10 from .store import JsonStore
11
12
13 class TaskCancelled(Exception):
14     pass
15
16
17 @dataclass
18 class AsyncTask:
19     task_id: str
20     user_id: str
21     session_id: str
22     name: str
23     status: str
24     cancel_event: Event
25     future: Future | None = None
26     result: Any = None
27
28
29 class AsyncTaskManager:
30     """In-process async task demo with completion events and cooperative cancellation."""
31
32     def __init__(self, store: JsonStore) -> None:
33         self.store, self.lock = store, RLock()
34         self.executor = ThreadPoolExecutor(max_workers=2, thread_name_prefix="agent-task")
35         self.tasks: dict[str, AsyncTask] = {}
36
```

```

37 def submit(self, user_id: str, session_id: str, name: str, work: Callable[[Event], Any]) -> dict[str, Any]:
38     # 任务立即返回 task_id, 耗时 work 在线程池执行, 不阻塞主 Agent Loop。
39     task = AsyncTask(uuid4().hex[:10], user_id, session_id, name, "queued", Event())
40     with self.lock:
41         self.tasks[task.task_id] = task
42
43     def runner() -> Any:
44         task.status = "running"
45         if task.cancel_event.is_set():
46             raise TaskCancelled()
47         return work(task.cancel_event)
48
49     def complete(future: Future) -> None:
50         # 后台任务结束后写入所属 session 的事件; 下次交互可读取并通知用户。
51         try:
52             task.result = future.result()
53             task.status = "completed"
54             event = {"type": "task_completed", "task_id": task.task_id, "name": name, "result": task.result}
55         except (TaskCancelled, Exception) as exc:
56             task.status = "cancelled" if isinstance(exc, TaskCancelled) or task.cancel_event.is_set() else "failed"
57             event = {"type": f"task_{task.status}", "task_id": task.task_id, "name": name, "error": str(exc)}
58         self.store.add_event(user_id, session_id, event)
59
60     task.future = self.executor.submit(runner)
61     task.future.add_done_callback(complete)
62     return {"task_id": task.task_id, "status": "queued", "message": "Task started in background; check task_control
> later."}
63
64 def cancel(self, task_id: str, user_id: str, session_id: str) -> dict[str, Any]:
65     # Python 不能安全强杀运行中的线程, 所以采用 cancel_event 的协作式取消。
66     task = self._owned(task_id, user_id, session_id)
67     task.cancel_event.set()
68     if task.future:
69         task.future.cancel()
70     return {"task_id": task_id, "status": "cancel_requested"}
71
72 def status(self, task_id: str, user_id: str, session_id: str) -> dict[str, Any]:
73     task = self._owned(task_id, user_id, session_id)
74     return {"task_id": task.task_id, "name": task.name, "status": task.status, "result": task.result}
75
76 def _owned(self, task_id: str, user_id: str, session_id: str) -> AsyncTask:
77     # task_id 即使猜中, 也不能跨 user/session 查询或取消。
78     task = self.tasks.get(task_id)
79     if not task or (task.user_id, task.session_id) != (user_id, session_id):
80         raise ValueError("task not found in current session")
81     return task
82
83 def shutdown(self) -> None:
84     self.executor.shutdown(wait=True)

```

```
85
86
87 def delayed_search(query: str, delay_seconds: float, cancel_event: Event) -> dict[str, Any]:
88     # 用可取消的 mock 延迟模拟真实的长耗时网络工具。
89     deadline = time() + max(0.05, min(delay_seconds, 2.0))
90     while time() < deadline:
91         if cancel_event.is_set():
92             raise TaskCancelled()
93         sleep(0.02)
94     return {"query": query, "results": [f"Background mock result for: {query}"]}
```

runtime.py

它负责什么：Agent 的核心大脑。它把模型、工具、记忆和 session 状态串成完整循环。

建议怎么读：按 run → _run_claimed → _context → _compress → _explicit_memory 的顺序看。

本文件最重要的理解：最关键循环是：组装 Context → 调模型 → 有工具就执行并把结果回传 → 没工具就返回最终答案。

完整代码（带行号）

```
1 from __future__ import annotations
2
3 import json
4 import re
5 from threading import RLock
6 from typing import Any
7
8 from .store import JsonStore
9 from .tools import ToolRegistry
10 from .types import AgentResult
11
12 SYSTEM_PROMPT = """You are a concise helpful assistant. Use a tool whenever it is needed. Do not invent tool results. If a
> tool result answers the request, explain it to the user."""
13
14
15 class AgentRuntime:
16     def __init__(self, client: Any, store: JsonStore, tools: ToolRegistry, max_turns: int = 5, recent_messages: int = 10) ->
> None:
17         self.client, self.store, self.tools = client, store, tools
18         self.max_turns, self.recent_messages = max_turns, recent_messages
19         self.lock = RLock()
20
21     def run(self, user_id: str, session_id: str, user_input: str) -> AgentResult:
22         # 先占用 session。若已被另一个请求占用，则只入队，不并发修改上下文。
23         if not self.store.claim_session(user_id, session_id, user_input):
24             return AgentResult(session_id, "当前 session 正在处理上一条消息，新消息已进入队列。", [{"event": "input_queued", "input": user_input}])
25         return self._run_claimed(user_id, session_id, user_input)
26
27     def drain_next(self, user_id: str, session_id: str) -> AgentResult | None:
28         """Process one queued user input after the active request has completed."""
29         queued = self.store.claim_next_input(user_id, session_id)
30         return self._run_claimed(user_id, session_id, queued) if queued is not None else None
31
32     def _run_claimed(self, user_id: str, session_id: str, user_input: str) -> AgentResult:
33         session = self.store.session(user_id, session_id)
34         trace: list[dict[str, Any]] = []
```

```

35     try:
36         # “记住……” 属于显式写入长期记忆，避免自动提取造成错误记忆。
37         remembered = self._explicit_memory(user_id, user_input)
38         session["messages"].append({"role": "user", "content": user_input})
39         self._compress(session, trace)
40
41     for turn in range(1, self.max_turns + 1):
42         # 每轮都重新组装 Context：记忆/摘要/最近消息会影响模型下一步决策。
43         messages = self._context(session, user_id, user_input)
44         trace.append({"event": "llm_request", "turn": turn, "message_count": len(messages)})
45         try:
46             reply = self.client.chat(messages, self.tools.schemas())
47         except Exception as exc:
48             return AgentResult(session_id, f"模型调用失败: {exc}", trace + [{"event": "llm_error", "error": str(exc)}])
49
50     if not reply.tool_calls:
51         # 没有工具调用即为最终回答，结束本轮 Agent Loop。
52         answer = reply.content or (f"我已记住: {remembered}" if remembered else "已完成。")
53         session["messages"].append({"role": "assistant", "content": answer})
54         return AgentResult(session_id, answer, trace)
55
56     session["messages"].append({"role": "assistant", "content": reply.content or "", "tool_calls": [{"id": c.id,
> "type": "function", "function": {"name": c.name, "arguments": json.dumps(c.arguments, ensure_ascii=False)}} for c in
> reply.tool_calls]})
57     for call in reply.tool_calls:
58         try:
59             # 工具异常会转成 tool result 回传给模型，而不是让整个 Agent 崩溃。
60             result: Any = self.tools.execute(call.name, call.arguments, {"user_id": user_id, "session_id":
> session_id})
61             trace.append({"event": "tool_success", "tool": call.name, "arguments": call.arguments, "result":
> result})
62         except Exception as exc:
63             result = {"error": str(exc)}
64             trace.append({"event": "tool_error", "tool": call.name, "error": str(exc)})
65             session["messages"].append({"role": "tool", "tool_call_id": call.id, "content": json.dumps(result,
> ensure_ascii=False)})
66         self._compress(session, trace)
67
68     return AgentResult(session_id, "工具调用轮次已达到上限，请缩小问题后重试。", trace, stopped_by_limit=True)
69 finally:
70     # 无论成功、失败还是达到轮次上限，都释放 session，保证排队消息可继续执行。
71     self.store.save_session(session)
72     self.store.finish_session(user_id, session_id)
73
74 def _context(self, session: dict[str, Any], user_id: str, query: str) -> list[dict[str, Any]]:
75     messages: list[dict[str, Any]] = [{"role": "system", "content": SYSTEM_PROMPT}]
76     memories = self.store.recall(user_id, query)
77     if memories:
78         messages.append({"role": "system", "content": "Relevant user memory (may be stale; current user instruction

```



```

> wins):\n- " + "\n- ".join(memories)))
79     if session["summary"]:
80         # 摘要要是旧历史的压缩表示，最近消息仍保留原文以支持自然追问。
81         messages.append({"role": "system", "content": "Structured session summary:\n" + session["summary"]})
82         messages.extend(session["messages"][-self.recent_messages:])
83         return messages
84
85     def _compress(self, session: dict[str, Any], trace: list[dict[str, Any]]) -> None:
86         messages = session["messages"]
87         if len(messages) <= self.recent_messages * 2:
88             return
89         old, session["messages"] = messages[-self.recent_messages:], messages[-self.recent_messages:]
90         try:
91             # 第二版优先让模型输出四类结构化状态，避免普通摘要遗漏“未完成事项”。
92             session["summary"] = self.client.summarize(session["summary"], old)
93             trace.append({"event": "structured_summary", "strategy": "llm_json"})
94         except Exception as exc:
95             # API 不可用或 JSON 不合法时，使用确定性文本摘要作为兜底。
96             facts = [f"{item['role']}: {item['content'][:160]}" for item in old if item["role"] in {"user", "assistant"} and
> item.get("content")]
97             session["summary"] = (session["summary"] + "\n" + "\n".join(facts))[-2000:]
98             trace.append({"event": "structured_summary", "strategy": "fallback", "error": str(exc)})
99
100     def _explicit_memory(self, user_id: str, text: str) -> str | None:
101         match = re.search(r"(?:请)?记住[: , \s]*(.+)", text)
102         if match:
103             memory = match.group(1).strip("。！！")
104             if memory:
105                 self.store.remember(user_id, memory)
106             return memory
107         return None

```